

TRIGGER
docker run / kubectl apply
/ python -m src.runner

Stage 1 – Preflight Check

Verify Python ≥ 3.9
Verify PyTorch
Check
torch.cuda.is_available()

Probe nvidia-smi
(GPU name, driver, memory)

Probe pynvml (GPU count)
FAIL FAST if critical check
fails

Stage 2 – Environment Capture

Python, PyTorch, CUDA,
cuDNN, driver versions

Docker container ID,
OS, hostname

pip freeze → SHA-256
→ environment.json

Stage 3 – Benchmark Execution (src/runner.py)

Read benchmark_config.yaml
workloads · batch_sizes ·
modes · repeats · seed

FOR EACH (workload × mode × batch_size × repeat)

Set deterministic seed
(torch, numpy, CUDA, cuDNN)

Instantiate workload
(ResNet-50 / BERT-base)

Warmup iterations
(fill CUDA caches)

Start GPU collector
(pynvml, 500ms polls)

Timed iterations
(CUDA events, sub-ms)

Stop collector
Compute metrics
Write latency CSV

Write summary CSV +
manifest
Auto-log to history DB
(SQLite)

Stage 4 – Report Generation

Preprocessor
aggregate stats, CV,
noisy-run flags

Cost Analysis
throughput_per_dollar
cost_per_1k_samples

Visualizer
7 chart types (PNG)

HTML Report
Jinja2 + base64 charts

Stage 5 – GPU Recommendation

Load aggregated results

Score GPUs
normalise to 0-1

Apply constraints
(budget, SLA, memory)

Rank feasible GPUs
Generate reasoning

Write recommendation.json

DONE – All artifacts in
results/